

Project Documentation

Student Attendance Tracking Application

Md. Abu Bakar Siddik (ID:222-35-1188)

Md. Tanver Bahar Jehad (ID: 222-35-1106)

Sadia Alam Mim (ID: 222-35-1110)

Member Role

Md. Tanver Bahar Jehad (222-35-1106): Admin Dashboard

Md. Abu Bakar Siddik (222-35-1188): Teacher Dashboard

Sadia Alam Mim (222-35-1110): Student Dashboard

Contents

Chapter 1: Introduction.....	3
1. Project Planning and Initiation.....	3
Feasibility Study (Step-by-Step)	3
2. Target User Profile and Tentative Elicitation Process.....	3
3. System Requirements	4
4. Project Scheduling.....	4
Chapter 2: Design and Implementation.....	4
1. Functional Requirements	4
2. Non-Functional Requirements	4
3. Object-oriented System Design using UML and the codes.....	5
Chapter 3: Software Testing	35
1. Testing Features	35
2. Testing Strategies	35
3. System Testing.....	35
Chapter 4: Deployment and Maintenance	36
Chapter 5: User Manual	36
Chapter 6: Project Summary	37

Chapter 1: Introduction

1. Project Planning and Initiation

Feasibility Study (Step-by-Step)

Phase 1: Preliminary Analysis & Project Scope Definition

The Student Attendance Tracking Application aims to automate and simplify the process of marking, storing, and analyzing attendance records for educational institutions. The system will cater to administrators, teachers, and students with dedicated dashboards and functionalities.

Phase 2: Market Feasibility Analysis

With the increasing adoption of digital tools in education, a market-ready attendance tracking solution can address inefficiencies in manual processes. Similar systems exist, but our application provides role-based access, PDF export, and chart-based analytics, offering a competitive advantage.

Phase 3: Technical Feasibility Analysis

The system is developed using Laravel 11 (PHP 8.2+) for backend, MySQL for database, Bootstrap 5 and Tailwind CSS for frontend styling, and Laravel Sanctum for authentication. These technologies are mature, open-source, and well-supported.

Phase 4: Financial Feasibility Analysis

The project uses free and open-source frameworks, reducing development costs significantly. Deployment can be done on low-cost hosting solutions, making it financially viable for small to medium institutions.

2. Target User Profile and Tentative Elicitation Process

Target Users:

- Administrators: Manage user roles, view global statistics.
- Teachers: Mark attendance, view class reports.
- Students: View personal attendance records and download reports.

Elicitation Process: Conducted interviews with teachers, administrators, and students; collected feedback through surveys; and analyzed existing attendance processes to gather requirements.

3. System Requirements

Hardware: Minimum dual-core processor, 4GB RAM, 20GB storage.

Software: Laravel 11, PHP 8.2+, MySQL, Bootstrap 5, Tailwind, Laravel Sanctum, Chart.js, Laravel DomPDF.

4. Project Scheduling

The project was planned over 8 weeks covering planning, development, testing, and deployment.

A Gantt chart (to be inserted) will show timelines for each module.

Risk Management:

- Risk: Data loss – Mitigation: Regular backups.
- Risk: Unauthorized access – Mitigation: Role-based access control and authentication.
- Risk: Deadline delay – Mitigation: Agile iterations and regular progress tracking.

Chapter 2: Design and Implementation

1. Functional Requirements

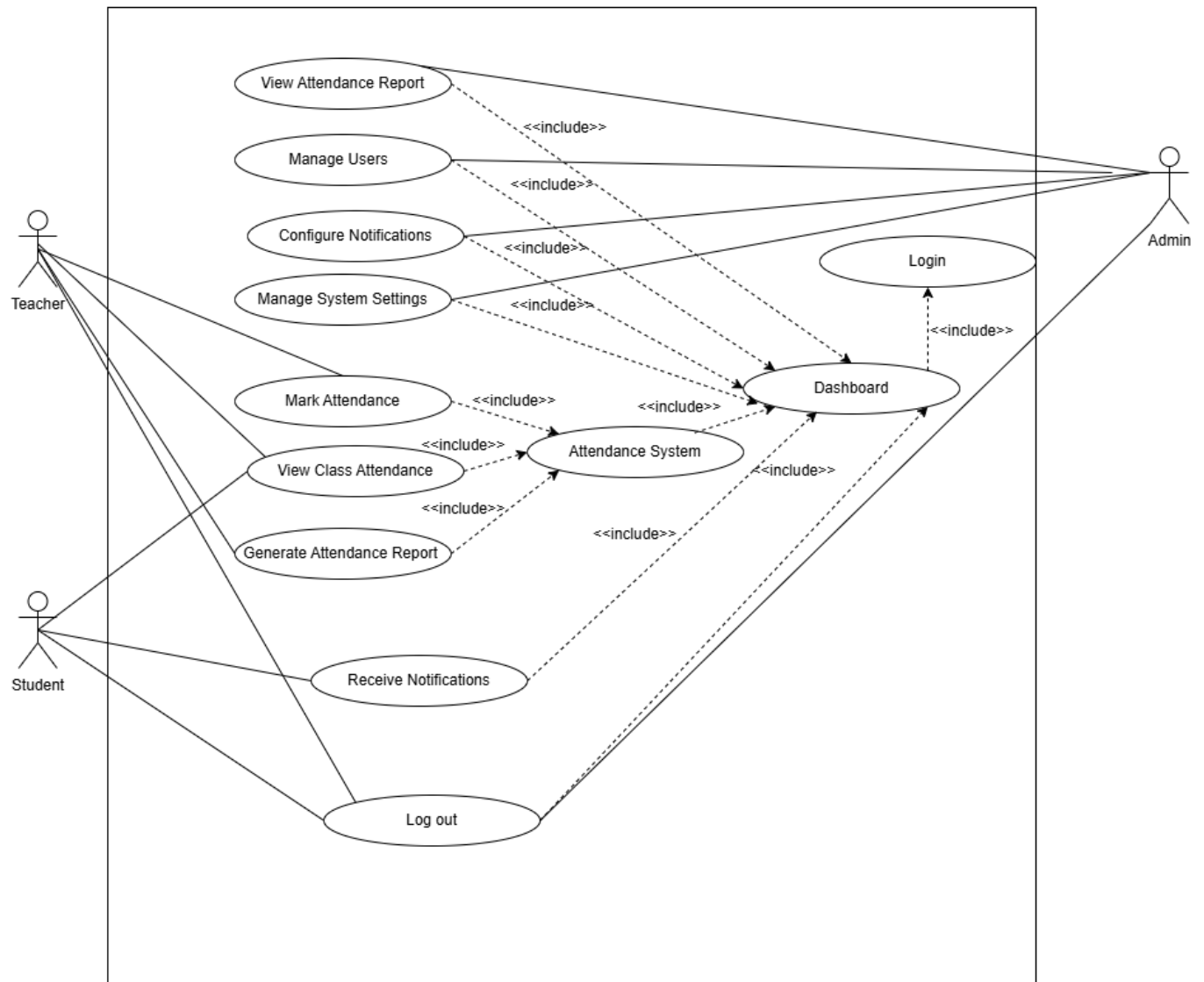
- User authentication and role-based dashboards
- Attendance marking for teachers
- Attendance viewing for students
- Report generation and PDF export
- Data visualization using charts
- Admin management of users and data

2. Non-Functional Requirements

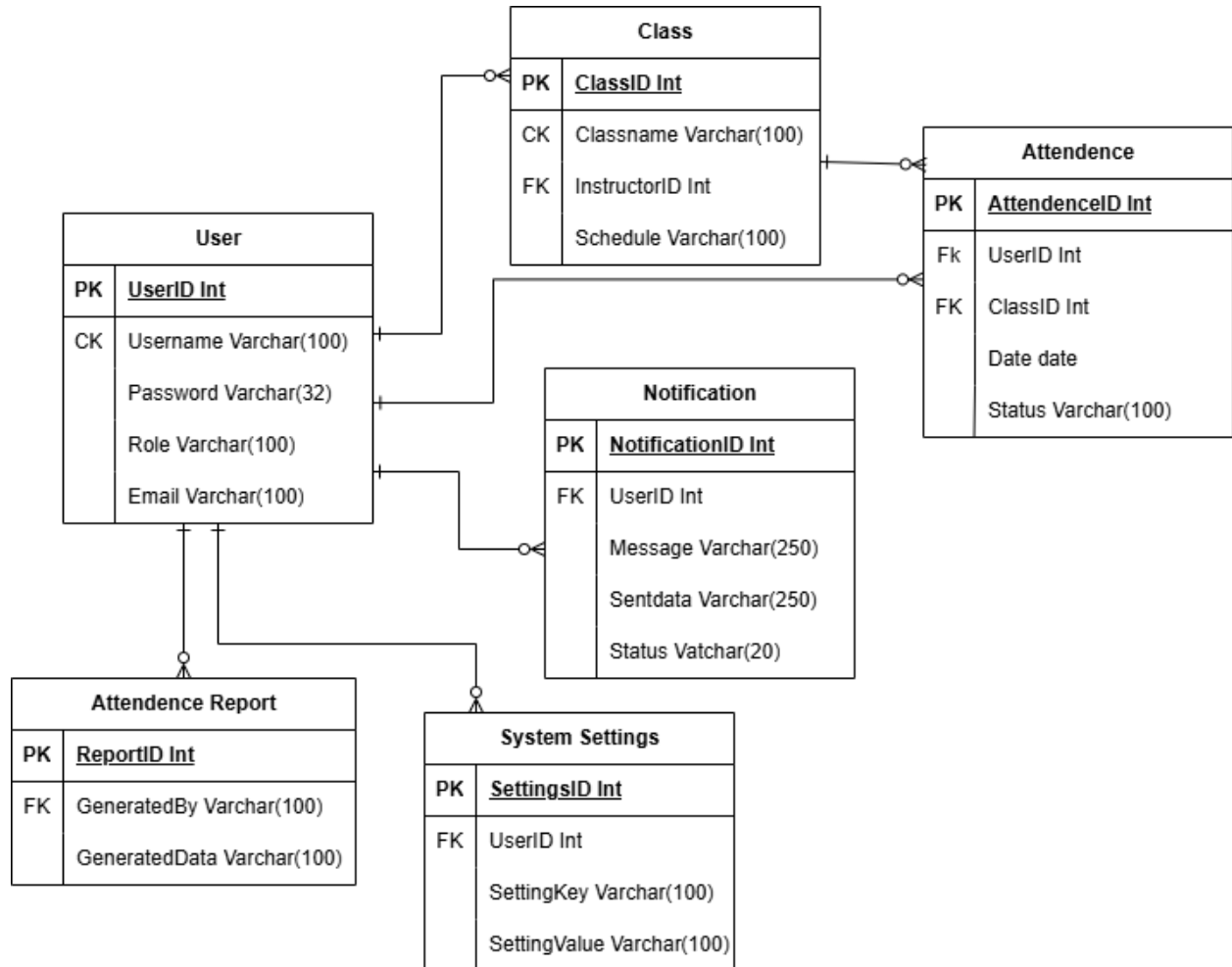
- Security: Laravel Sanctum authentication, CSRF protection
- Performance: Optimized queries, caching
- Usability: Responsive UI
- Scalability: Can support multiple institutions
- Maintainability: MVC architecture

3. Object-oriented System Design using UML and the codes

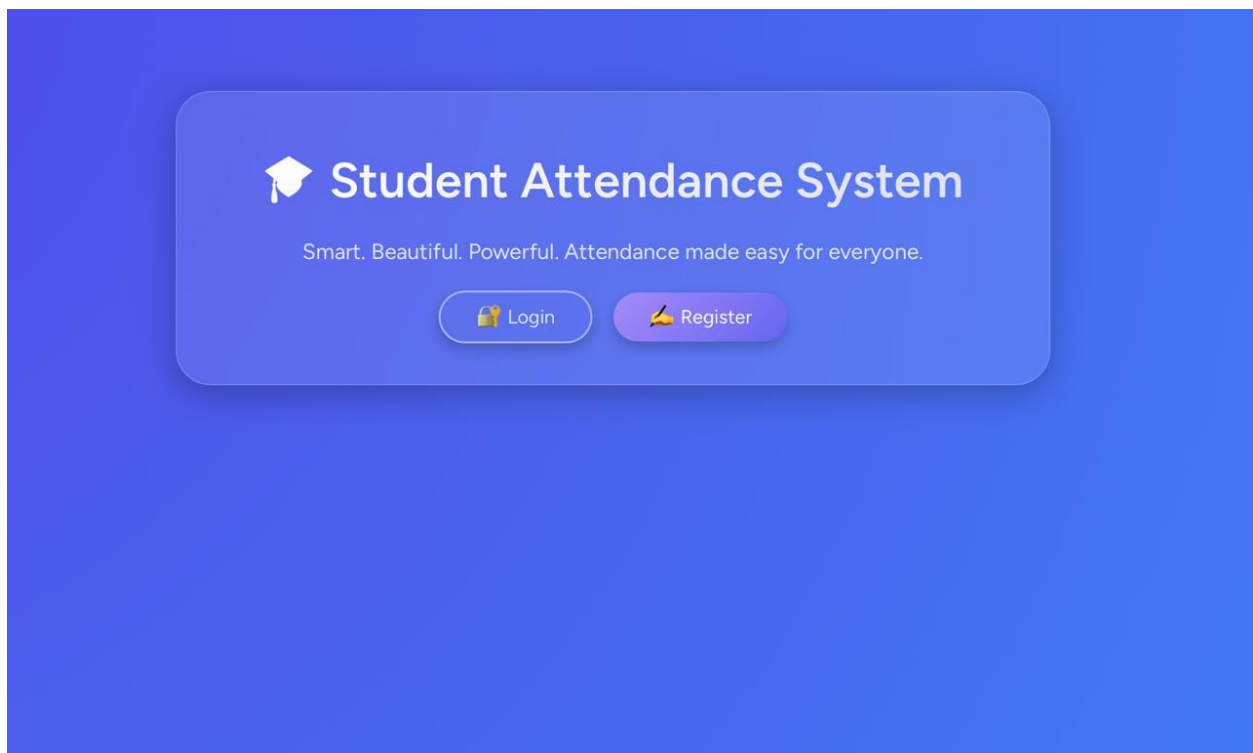
Use Case



ER Diagram



Home page



Main Home page for our web application. It's available for all type of user for log in on that application

Admin Dashboard

222-35-1106

Welcome, Jabir 🙌

Monitor and manage key system metrics.

JA

Total Teachers

2



Total Students

4



Today's Attendance Rate

0%



Attendance Trends by Day

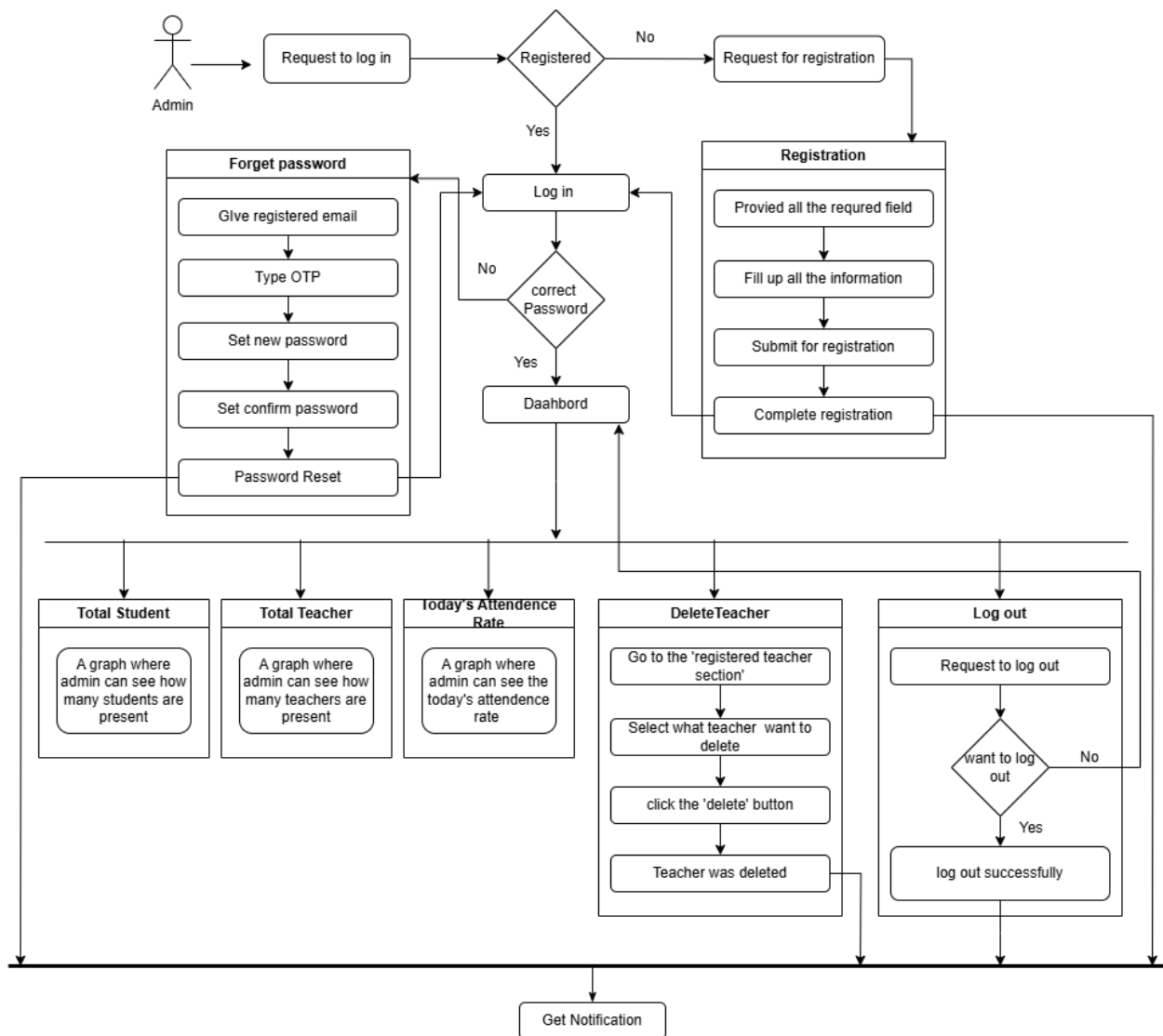


Present vs Absent (Today)

Present Absent

Registered Teachers

ID	Name	Email	Delete
3	Sifat	sifat_teach@gmail.com	Delete
7	Ashraf	ashraf_teach@yahoo.com	Delete



Block diagram of Admin

Key Admin Dashboard Technologies Summary:

Category	Technologies
Backend	Laravel 11, PHP 8.2+, Eloquent ORM, Carbon
Frontend	Bootstrap 5, Chart.js, Alpine.js, Font Awesome
Build	Vite, Laravel Vite Plugin
Database	MySQL, Laravel Migrations
Auth	Laravel Sanctum, Spatie Permissions
Charts	Chart.js (Line & Pie charts)
Styling	Custom CSS, Bootstrap, Tailwind

Backend Architecture for Admin Dashboard

The AdminDashboardController is responsible for processing admin dashboard requests and preparing data for the view.

Key Components:

```
class AdminDashboardController extends Controller
{
    public function index()
    {
        // Data processing logic here
    }
}
```

1.Data Processing Flow:

A. Date Calculations:

```
$today = Carbon::today()->toDateString(); // Current date (YYYY-MM-DD)
$month = Carbon::now()->month;           // Current month (1-12)
$year = Carbon::now()->year;              // Current year
```

B. User Statistics:

```
$totalTeachers = User::role('teacher')->count(); // Count teachers
$totalStudents = User::role('student')->count(); // Count students
```

C. Attendance Calculations:

```
$presentToday = Attendance::whereDate('date', $today)
    ->where('status', 'present')->count();
$absentToday = Attendance::whereDate('date', $today)
    ->where('status', 'absent')->count();

$totalToday = $presentToday + $absentToday;
$attendanceRate = $totalToday > 0 ?
    round(($presentToday / $totalToday) * 100) : 0;
```

D. Chart Data Generation:

```
$labels = [];
$presentCounts = [];

for ($d = 1; $d <= 30; $d++) {
    $date = Carbon::createFromDate($year, $month, $d)->format('Y-m-d');
    $labels[] = $d; // Day number (1-30)
    $presentCounts[] = Attendance::whereDate('date', $date)
        ->where('status', 'present')->count();
}
```

2. Model Layer

A. User Model (User.php):

```
class User extends Authenticatable
{
    use HasRoles; // Spatie Permission trait

    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    public function detectRoleFromEmail(): string
    {
        // Role detection logic based on email patterns
    }
}
```

Key Features:

- **Eloquent Relationships** - Links to Student model
- **Date Casting** - Automatic date format handling
- **Status Enum** - Present/Absent/Late status tracking

B. Attendance Model (Attendance.php):

```
class Attendance extends Model
{
    protected $fillable = [
        'student_id',
        'date',
        'status',
        'remarks'
    ];

    protected $casts = [
        'date' => 'date', // Automatic date casting
    ];

    public function student()
    {
        return $this->belongsTo(Student::class);
    }
}
```

Key Features:

- **Eloquent Relationships** - Links to Student model
- **Date Casting** - Automatic date format handling
- **Status Enum** - Present/Absent/Late status tracking

C. Student Model (Student.php):

```
class Student extends Model
{
    protected $fillable = [
        'name',
        'roll_number',
        'class',
        'email',
        'phone',
    ];

    public function attendances()
    {
        return $this->hasMany(Attendance::class);
    }
}
```

3. Database Schema

A. Users Table:

```
CREATE TABLE users (  
    id BIGINT PRIMARY KEY,  
    name VARCHAR(255),  
    email VARCHAR(255) UNIQUE,  
    password VARCHAR(255),  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
);
```

B. Attendances Table:

```
CREATE TABLE attendances (  
    id BIGINT PRIMARY KEY,  
    student_id BIGINT FOREIGN KEY,  
    date DATE,  
    status ENUM('present', 'absent', 'late') DEFAULT 'present',  
    remarks TEXT NULL,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP,  
    UNIQUE(student_id, date) -- Prevents duplicate entries  
);
```

C. Students Table:

```
CREATE TABLE students (  
    id BIGINT PRIMARY KEY,  
    name VARCHAR(255),  
    roll_number VARCHAR(255) UNIQUE,  
    class VARCHAR(255),  
    email VARCHAR(255) UNIQUE,  
    phone VARCHAR(255) NULL,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
);
```

4. Routing Layer

Admin Dashboard Route:

```
Route::get('/redirect', function () {
    $user = Auth::user();

    if ($user->hasRole('admin')) {
        return redirect()->route('admin.dashboard');
    }
    // ... other role checks
})->middleware(['auth'])->name('redirect');
```

Role-based Redirection:

```
Route::get('/redirect', function () {
    $user = Auth::user();

    if ($user->hasRole('admin')) {
        return redirect()->route('admin.dashboard');
    }
    // ... other role checks
})->middleware(['auth'])->name('redirect');
```

5. Security Implementation

A. Authentication:

- **Laravel Sanctum** - API token authentication
- **Session-based Auth** - Web interface authentication
- **Password Hashing** - Secure password storage

B. Authorization:

- **Spatie Laravel Permission** - Role-based access control
- **Middleware Protection** - Route-level security
- **Role Verification** - `$user->hasRole('admin')`

C. Data Protection:

- **CSRF Protection** - Cross-site request forgery prevention
- **SQL Injection Prevention** - Eloquent ORM protection
- **Input Validation** - Data sanitization

6. Performance Optimizations

A. Database Queries:

- **Efficient Counting** - `count()` instead of `get()->count()`
- **Date Indexing** - Optimized date-based queries
- **Relationship Loading** - Eager loading when needed

B. Caching Strategy:

- **Query Caching** - Laravel's built-in caching
- **View Caching** - Blade template compilation
- **Route Caching** - Compiled route lists

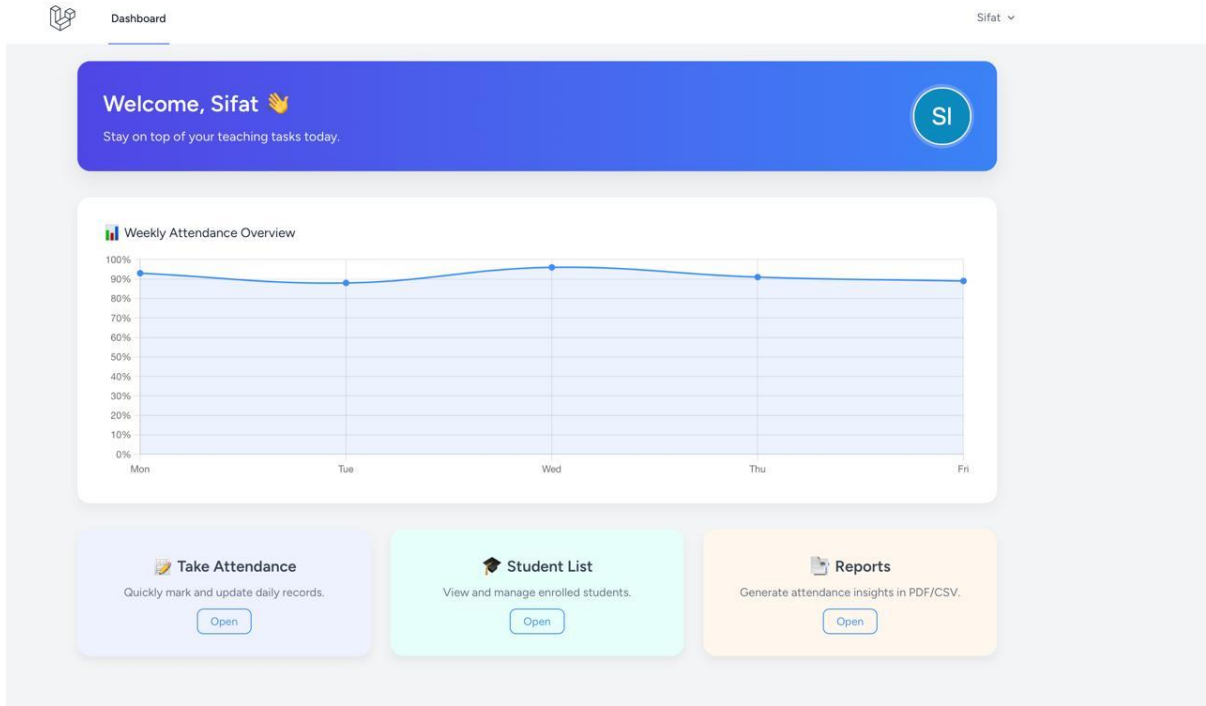
7. Error Handling

A. Exception Handling:

```
try {  
    $attendanceRate = $totalToday > 0 ?  
        round(($presentToday / $totalToday) * 100) : 0;  
} catch (Exception $e) {  
    $attendanceRate = 0;  
}
```

Teacher Dashboard

222-35-1188



Dashboard Sifat ▾

Take Attendance — Tuesday, July 15, 2025

Ali ID: 4	Absent ▾
Araf ID: 1	Present ▾
Hasib ID: 8	Present ▾
Omor ID: 2	Absent ▾
Cancel Open	

Attendance Report

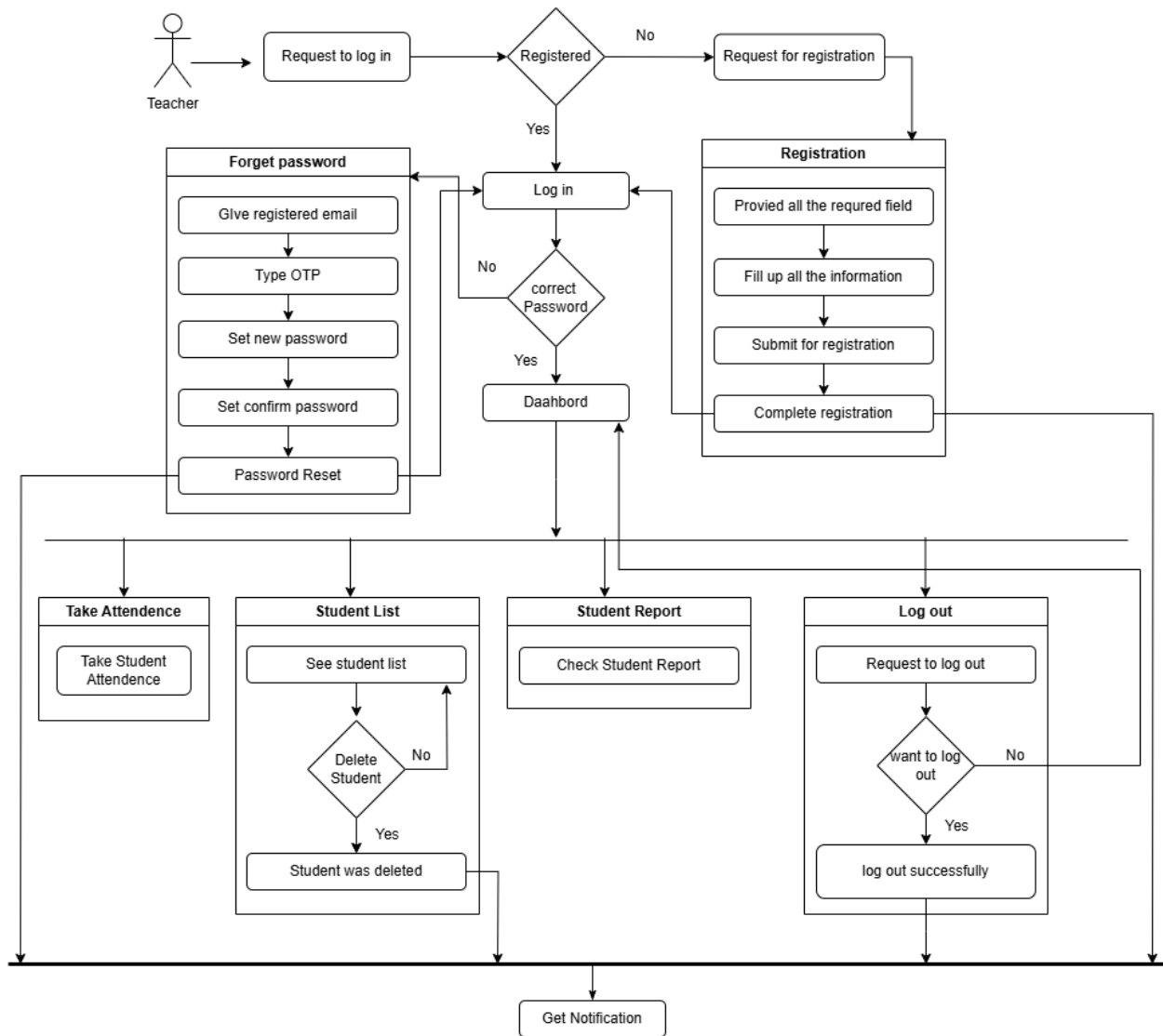
Start Date: 07/01/2025
End Date: 07/31/2025

Filter
CSV [Open](#)

Name	Total Days	Present	Absent	Attendance %
Ali	1	1	0	100%
Araf	1	1	0	100%
Hasib	1	0	1	0%
Omor	1	0	1	0%

Registered Students

ID	Name	Email	Action
2	Omor	omor_std@yahoo.com	Delete
4	Ali	ali_std@gmail.com	Delete
5	Araf	araf_std@yahoo.com	Delete
8	Hasib	hasib_std@yahoo.com	Delete



Block diagram of Teacher

Key Teacher Dashboard Technologies Summary:

Category	Technologies
Backend	Laravel 11, PHP 8.2+, Eloquent ORM, Carbon
Frontend	Bootstrap 5, Chart.js, Alpine.js, Font Awesome
Build	Vite, Laravel Vite Plugin
Database	MySQL, Laravel Migrations
Auth	Laravel Sanctum, Spatie Permissions
Charts	Chart.js (Line chart)
Styling	Custom CSS, Bootstrap, Tailwind
PDF	Laravel DomPDF

Backend Architecture for Teacher Dashboard

1. Routing Layer

```
Route::get('/teacher-dashboard', function () {
    return view('dashboards.teacher');
})->middleware(['auth', 'role:teacher'])->name('teacher.dashboard');
```

2.Role-based Redirection:

```
Route::get('/redirect', function () {
    $user = Auth::user();

    if ($user->hasRole('teacher')) {
        return redirect()->route('teacher.dashboard');
    }
    // ... other role checks
})->middleware(['auth'])->name('redirect');
```

3. Controller Layer

A. AttendanceController (Primary Controller for Teachers)

1. Create Attendance Form (create()):

```
public function create()
{
    $date = Carbon::today()->format('Y-m-d'); // 💎💎 force today's date
    $students = Student::all();

    $existingAttendance = Attendance::whereDate('date', $date)
        ->pluck('student_id')
        ->toArray();

    return view('attendances.create', compact('students', 'date', 'existingAttendance'));
}
```

Features:

- **Date Restriction** - Forces today's date only
- **Student List** - Retrieves all students for attendance marking
- **Duplicate Prevention** - Checks existing attendance records
- **Data Preparation** - Prepares data for the attendance form

2. Store Attendance Records (store()):

```
public function store(Request $request)
{
    $today = Carbon::today()->format('Y-m-d');

    $validated = $request->validate([
        'attendance' => 'required|array',
        'attendance.*.student_id' => 'required|exists:students,id',
        'attendance.*.status' => 'required|in:present,absent,late',
        'attendance.*.remarks' => 'nullable|string',
    ]);

    foreach ($validated['attendance'] as $record) {
        // 🚫 Prevent duplicate entries for today
        $existing = Attendance::where('student_id', $record['student_id'])
            ->whereDate('date', $today)
            ->first();

        if ($existing) {
            continue; // skip this one
        }

        Attendance::create([
            'student_id' => $record['student_id'],
            'status' => $record['status'],
            'remarks' => $record['remarks'] ?? null,
            'date' => $today,
        ]);
    }

    return redirect()->route('attendances.index', ['date' => $today])
        ->with('success', 'Attendance recorded successfully.');
```

Features:

- **Input Validation** - Validates attendance data structure
- **Duplicate Prevention** - Prevents multiple entries for same student/day
- **Batch Processing** - Handles multiple student records at once
- **Success Feedback** - Provides user feedback after submission

3. View Attendance Records (index()):

```
public function index()
{
    $date = request('date', Carbon::today()->format('Y-m-d'));

    $attendances = Attendance::with('student')
        ->whereDate('date', $date)
        ->orderBy('created_at', 'desc')
        ->paginate(10);

    return view('attendances.index', compact('attendances', 'date'));
}
```

Features:

- **Date Filtering** - Shows attendance for specific date
- **Eager Loading** - Loads student data efficiently
- **Pagination** - Handles large datasets
- **Default Date** - Uses today's date if none specified

4. Generate Reports (report()):

```
public function report()
{
    $month = request('month', Carbon::now()->format('Y-m'));

    $students = Student::with(['attendances' => function ($query) use ($month) {
        $query->whereYear('date', Carbon::parse($month)->year)
            ->whereMonth('date', Carbon::parse($month)->month);
    }])->get();

    return view('attendances.report', compact('students', 'month'));
}
```

Features:

- **Monthly Filtering** - Shows data for specific month
- **Eager Loading** - Efficiently loads attendance relationships
- **Flexible Date Range** - Allows month selection
- **Comprehensive Data** - Provides complete student attendance overview

5. PDF Export (exportPdf()):

```
public function exportPdf()
{
    $student = Auth::user();
    $month = request('month', Carbon::now()->format('Y-m'));

    $records = Attendance::whereHas('student', function ($query) use ($student) {
        $query->where('email', $student->email);
    })->whereYear('date', Carbon::parse($month)->year)
        ->whereMonth('date', Carbon::parse($month)->month)
        ->orderBy('date')
        ->get();

    $pdf = Pdf::loadView('reports.attendance-pdf', [
        'student' => $student,
        'records' => $records,
        'month' => $month,
    ]);

    return $pdf->download("Attendance_Report_{$student->name}_{$month}.pdf");
}
```

Features:

- **PDF Generation** - Creates downloadable PDF reports
- **User-specific Data** - Filters data for logged-in user
- **Monthly Reports** - Generates month-specific reports
- **File Download** - Provides downloadable PDF files

B. StudentController (Student Management)

1. List Students (index()):

```
public function index()
{
    $students = Student::latest()->paginate(10);
    return view('students.index', compact('students'));
}
```

Features:

- **Latest First** - Shows newest students first
- **Pagination** - Handles large student lists
- **Efficient Queries** - Optimized database queries

2. Create Student (create() & store()):

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'roll_number' => 'required|string|unique:students',
        'class' => 'required|string|max:255',
        'email' => 'required|email|unique:students',
        'phone' => 'nullable|string|max:20',
    ]);

    Student::create($validated);

    return redirect()->route('students.index')
        ->with('success', 'Student created successfully.');
```

Features:

- **Input Validation** - Validates all student data
- **Unique Constraints** - Ensures roll number and email uniqueness
- **Success Feedback** - Provides user confirmation
- **Data Integrity** - Maintains database consistency

3. Update Student (edit() & update()):

```
public function update(Request $request, Student $student)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'roll_number' => 'required|string|unique:students,roll_number,' . $student->id,
        'class' => 'required|string|max:255',
        'email' => 'required|email|unique:students,email,' . $student->id,
        'phone' => 'nullable|string|max:20',
    ]);

    $student->update($validated);

    return redirect()->route('students.index')
        ->with('success', 'Student updated successfully.');
```

Features:

- **Route Model Binding** - Automatic student retrieval
- **Unique Validation** - Excludes current record from uniqueness check
- **Data Update** - Efficient record updating
- **User Feedback** - Success confirmation

C. ProfileController (User Profile Management)

1. Edit Profile (edit()):

```
public function edit(Request $request): View
{
    return view('profile.edit', [
        'user' => $request->user(),
    ]);
}
```

2. Update Profile (update()):

```
public function update(ProfileUpdateRequest $request): RedirectResponse
{
    $user = $request->user();
    $user->fill($request->validated());

    if ($user->isDirty('email')) {
        $user->email_verified_at = null;
    }

    $user->save();

    return Redirect::route('profile.edit')->with('status', 'profile-updated');
}
```

Features:

- **Form Request Validation** - Uses dedicated validation class
- **Email Verification Reset** - Resets verification when email changes
- **Dirty Checking** - Only updates changed fields
- **Status Feedback** - Provides update confirmation

4. Request Validation Layer

ProfileUpdateRequest:

```
class ProfileUpdateRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'lowercase',
                'email',
                'max:255',
                Rule::unique(User::class)->ignore($this->user()->id),
            ],
        ];
    }
}
```

5. Security Implementation

A. Authentication:

- **Laravel Sanctum** - API token authentication
- **Session-based Auth** - Web interface authentication
- **Password Hashing** - Secure password storage

B. Authorization:

- **Spatie Laravel Permission** - Role-based access control
- **Middleware Protection** - Route-level security
- **Role Verification** - `$user->hasRole('teacher')`

C. Data Protection:

- **CSRF Protection** - Cross-site request forgery prevention
- **Input Validation** - Data sanitization and validation
- **SQL Injection Prevention** - Eloquent ORM protection
- **Duplicate Prevention** - Prevents multiple attendance entries

6. Error Handling

A. Validation Errors:

```
$validated = $request->validate([
    'attendance' => 'required|array',
    'attendance.*.student_id' => 'required|exists:students,id',
    'attendance.*.status' => 'required|in:present,absent,late',
    'attendance.*.remarks' => 'nullable|string',
]);
```

Student Dashboard

222-35-1110

Hello, Omor 🙌

Your attendance progress and tools are below.



Monthly Attendance Summary

You've attended -- out of -- days this month.

[View Detailed Report](#)

Attendance History

Track your past attendance records.

[Open](#)

Edit Profile

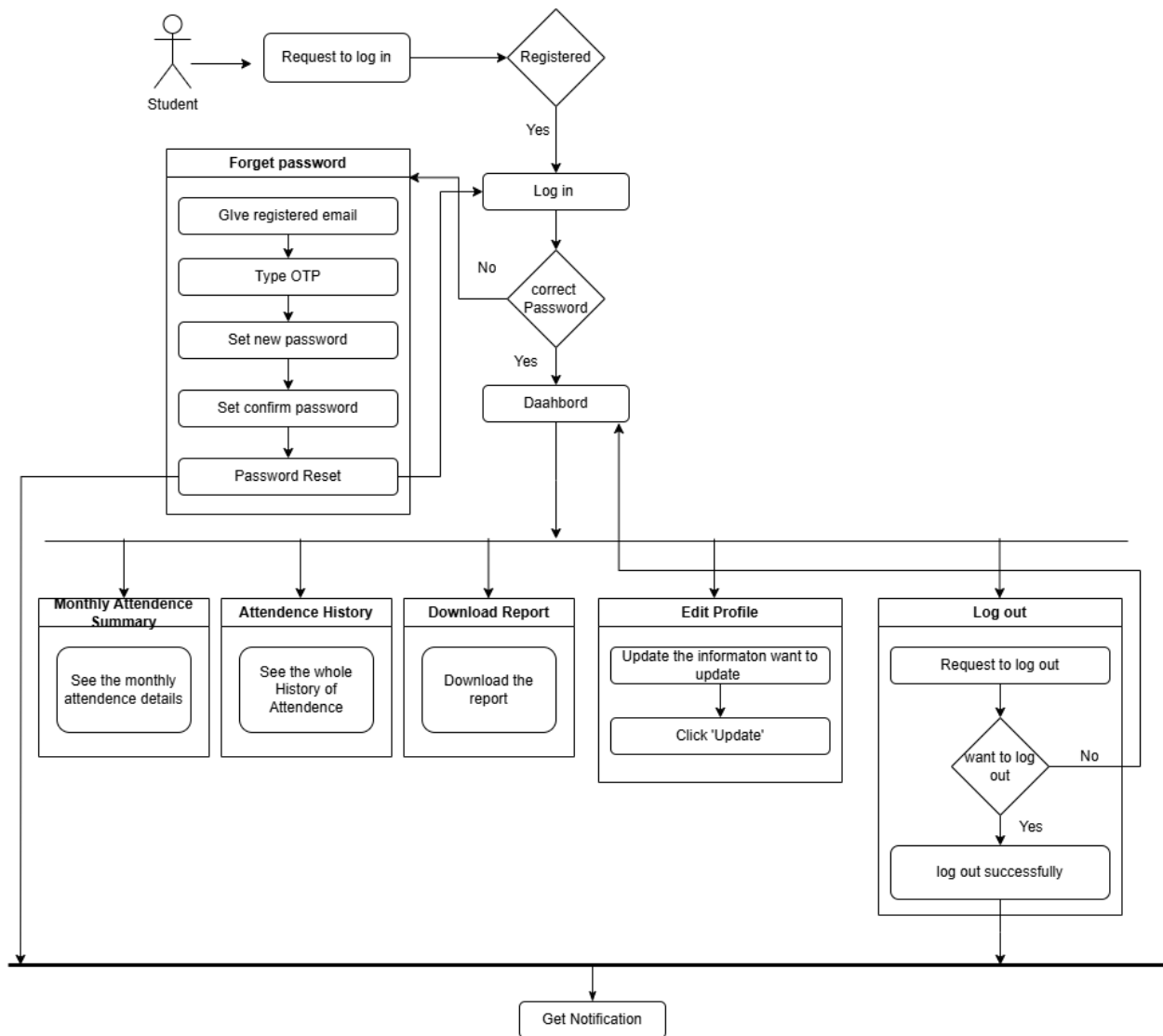
Update your personal details here.

[Open](#)

Download Report

Export your monthly attendance report.

[Download PDF](#)



Block diagram of Student

Key Student Dashboard Technologies Summary:

Category	Technologies
Backend	Laravel 11, PHP 8.2+, Eloquent ORM, Carbon
Frontend	Bootstrap 5, Alpine.js, Font Awesome
Build	Vite, Laravel Vite Plugin
Database	MySQL, Laravel Migrations
Auth	Laravel Sanctum, Spatie Permissions
PDF	Laravel DomPDF
Styling	Custom CSS, Bootstrap, Tailwind

Student Dashboard Backend Implementation:

Route Definition:

```
Route::get('/student-dashboard', function () {
    return view('dashboards.student');
})->middleware(['auth', 'role:student'])->name('student.dashboard');
```

Key Controllers Used:

A. AttendanceController (Student-specific methods):

1. View Personal Attendance (index()):

```
public function index()
{
    $date = request('date', Carbon::today()->format('Y-m-d'));

    $attendances = Attendance::with('student')
        ->whereDate('date', $date)
        ->orderBy('created_at', 'desc')
        ->paginate(10);

    return view('attendances.index', compact('attendances', 'date'));
}
```

Features:

- **Date Filtering** - View attendance for specific dates
- **Eager Loading** - Efficiently loads student data
- **Pagination** - Handles large datasets
- **Personal View** - Students see their own attendance records

2. Personal Attendance Report (report()):

```
public function report()
{
    $month = request('month', Carbon::now()->format('Y-m'));

    $students = Student::with(['attendances' => function ($query) use ($month) {
        $query->whereYear('date', Carbon::parse($month)->year)
            ->whereMonth('date', Carbon::parse($month)->month);
    }])->get();

    return view('attendances.report', compact('students', 'month'));
}
```

Features:

- **Monthly Filtering** - Shows data for specific month
- **Eager Loading** - Efficiently loads attendance relationships
- **Comprehensive Data** - Provides complete attendance overview
- **Student-specific** - Focuses on individual student data

3. Personal PDF Export (exportPdf()):

```
public function exportPdf()
{
    $student = Auth::user();
    $month = request('month', Carbon::now()->format('Y-m'));

    $records = Attendance::whereHas('student', function ($query) use ($student) {
        $query->where('email', $student->email);
    }->whereYear('date', Carbon::parse($month)->year)
        ->whereMonth('date', Carbon::parse($month)->month)
        ->orderBy('date')
        ->get();

    $pdf = Pdf::loadView('reports.attendance-pdf', [
        'student' => $student,
        'records' => $records,
        'month' => $month,
    ]);

    return $pdf->download("Attendance_Report_{$student->name}_{$month}.pdf");
}
```

Features:

- **Personal Data** - Filters records for logged-in student only
- **PDF Generation** - Creates downloadable PDF reports
- **Monthly Reports** - Generates month-specific reports
- **File Download** - Provides downloadable PDF files

B. ProfileController (Profile Management):

```
class ProfileController extends Controller
{
  public function edit(Request $request): View
  {
    return view('profile.edit', [
      'user' => $request->user(),
    ]);
  }

  public function update(ProfileUpdateRequest $request): RedirectResponse
  {
    $user = $request->user();
    $user->fill($request->validated());

    if ($user->isDirty('email')) {
      $user->email_verified_at = null;
    }

    $user->save();

    return Redirect::route('profile.edit')->with('status', 'profile-updated');
  }
}
```

Key Differences from Admin/Teacher Dashboards:

1. **No Dedicated Controller** - Uses shared controllers with role-based filtering
2. **Personal Data Focus** - Shows only student's own attendance data
3. **Read-Only Interface** - Students cannot modify attendance records
4. **Limited Actions** - Focuses on viewing and reporting rather than management
5. **Personal Reports** - PDF exports contain only student's own data
6. **No Charts** - Simpler interface without complex data visualization

Chapter 3: Software Testing

1. Testing Features

Tested features include authentication, attendance marking, PDF exports, and role-based dashboards.

2. Testing Strategies

Black-box testing for UI and functional modules; unit testing for backend logic.

3. System Testing

Test Case ID	Feature	Test Description	Test Data / Input	Expected Output	Actual Output	Status
TC-001	Login	Verify that valid credentials allow login	Email: jabir_admin1001@gmail.com, Password: jabir123	Redirect to Admin Dashboard	Redirected successfully	Pass
TC-002	Login	Verify that invalid credentials show error	Email: jabir_admin3003@gmail.com, Password: jabir123	"Invalid credentials" message	Error message displayed	Pass
TC-003	Attendance Marking	Teacher marks attendance for a class	Class list, Date: Today	Attendance saved in DB, Success message	Data saved and success shown	Pass
TC-004	Attendance Marking	Prevent duplicate attendance entry	Same class/date entered again	"Attendance already marked" message	Message displayed	Pass
TC-005	Report Generation	Teacher generates monthly	Month: July 2025	PDF file generated	PDF generated	Pass

		attendance report		with correct data		
TC-006	Student View	Student views personal attendance records	Logged in as student	Attendance list shown for logged-in student	Records displayed	Pass
TC-007	Role-Based Access	Prevent student from accessing Admin panel	Student tries /admin URL	"Access Denied" or redirect to dashboard	Access denied	Pass
TC-008	Security	CSRF protection works for attendance submission	Submit without token	Request blocked	Request blocked	Pass
TC-009	Data Validation	Prevent empty attendance submission	No students selected, click save	Error message shown	Error displayed	Pass

Chapter 4: Deployment and Maintenance

The system follows Agile methodology from the start. Deployment is done via shared hosting with regular updates and bug fixes. SRLC phases include pre-alpha, alpha, beta, release candidate, and production release.

Chapter 5: User Manual

Step 1: Login using provided credentials.

Step 2: Use the dashboard to access features based on your role.

Step 3: For teachers, mark attendance; for students, view/download reports.

Step 4: Logout when done.

Chapter 6: Project Summary

The Student Attendance Tracking Application successfully digitalizes and streamlines the attendance process, providing secure, role-based access, PDF reporting, and analytics. Built with modern web technologies, it is efficient, scalable, and user-friendly.